CAPSTONE PORTAL

Technical Design Document

Version 1.0 | March 2026

Technology	Language
Next.js + Firebase	TypeScript

1. Executive Summary

Capstone Portal is a full-stack web application designed to manage Final Year Projects (FYP) within a university environment. The system provides two distinct roles, Admin and Faculty, each with dedicated dashboards, data views, and management capabilities.

The platform is built using modern web technologies, including Next.js 15, TypeScript, Tailwind CSS v4, and Google Firebase. It supports real-time authentication, Firestore-backed persistent storage, and role-based access control enforced at both the UI and data layer.

Attribute	Value
Application Type	Web Portal (SPA with SSR)
Target Users	University Admin & Faculty Members
Primary Purpose	FYP Project & Faculty Management
Authentication	Firebase Auth (Email + Google OAuth)
Database	Cloud Firestore (NoSQL)
Deployment	Vercel / Any Node.js Host
Current Version	1.0 March 2026

2. System Architecture

2.1 High-Level Overview

The application follows a client-server architecture using Next.js App Router. Firebase acts as both the authentication provider and the backend database. There is no separate custom backend server; all data operations are performed directly from the client using the Firebase SDK over HTTPS.

> **Architecture: Client (Next.js) → Firebase Auth + Firestore → Google Cloud**

2.2 Architecture Layers

Layer	Technology / Description
Presentation	Next.js App Router + React + Tailwind CSS v4
State Management	React Context API (AuthContext)
Authentication	Firebase Authentication SDK
Data Access	Firestore SDK getDocs, addDoc, updateDoc, deleteDoc
Database	Cloud Firestore (NoSQL document database)
Hosting	Vercel (recommended) or any Node.js platform
CDN / Assets	Next.js built-in static optimization

2.3 Folder Structure

The project follows Next.js App Router conventions with a clear separation of concerns:

Path	Purpose
app/admin/	Admin-only pages: dashboard, projects, faculty, seed
app/faculty/	Faculty-only pages: dashboard, my projects
app/login/	Public login page (email + Google OAuth)
app/layout.tsx	Root layout wraps app in AuthProvider + Toaster
components/	Reusable UI: ProjectsTable, FacultyTable
context/AuthContext.tsx	Global auth state user, loading, logout
lib/firebase.ts	Firebase app + Firestore initialisation

lib/auth.ts	Auth helper functions: login, logout, getUserProfile
types/index.ts	TypeScript interfaces for all entities

3. Technology Stack

3.1 Frontend

Technology	Version / Details
Next.js	v15 App Router, file-based routing, SSR + CSR
React	v18 Functional components, hooks
TypeScript	v5 Full type safety across all files
Tailwind CSS	v4 Utility-first styling, no custom CSS files
lucide-react	v0.383 Icon library
react-hot-toast	v2 Toast notifications (success/error)

3.2 Backend / Cloud Services

Service	Purpose
Firebase Authentication	User login, session management, Google OAuth
Cloud Firestore	NoSQL document database for users and projects
Firebase SDK v12	Client-side access to Auth and Firestore

3.3 Development Tools

Tool	Purpose
Node.js	JavaScript runtime
npm	Package management
ESLint	Code linting
TypeScript Compiler	Type checking and build validation
VS Code	Recommended IDE

4. Authentication Flows

Authentication is handled entirely by Firebase Authentication. The system supports two sign-in methods and enforces role-based routing after every login.

4.1 Email & Password Login Flow

> **Step-by-step: Email + Password Authentication**

1. User enters email and password on /login page
2. signInWithEmailAndPassword() called via Firebase Auth SDK
3. Firebase returns a UserCredential with uid
4. getUserProfile(uid) fetches the Firestore document from users/{uid}
5. If document exists: profile returned with role field
6. If the document is missing: error is thrown. Profile not found. Contact admin.
7. Role check: admin → /admin/dashboard, faculty → /faculty/dashboard
8. window.location.href used for redirect (avoids auth timing issues)

4.2 Google OAuth Login Flow

> **Step-by-step: Google Sign-In Authentication**

9. User clicks the Sign in with Google button
10. signInWithPopup() opens the Google OAuth popup
11. Firebase returns credentials with Google uid
12. getUserProfile(uid) checks Firestore for an existing profile
13. First-time Google user: createUserProfile() auto-creates Firestore document with role = faculty
14. Returning user: existing profile loaded with saved role
15. Role-based redirect applied

4.3 Session Persistence

Firebase Auth automatically persists the user session in the browser using IndexedDB. On every page load, onAuthStateChanged fires, and the app calls getUserProfile() to restore the user's full profile, including their role.

> **onAuthStateChanged → getUserProfile(uid) → setUser(profile) → setLoading(false)**

4.4 Route Protection

Every protected page runs a useEffect that checks the auth state and redirects unauthorised users:

```
if (!loading && !user) router.push('/login');
if (!loading && user && user.role !== 'admin') router.push('/faculty/dashboard');
```

This ensures that faculty cannot access admin pages, and unauthenticated users are always redirected to /login.

4.5 Logout Flow

16. User clicks Sign out in the sidebar
17. signOut() called via Firebase Auth
18. setUser(null) clears the AuthContext state
19. window. location. href = '/login' forces a full page reload

5. Database Design

5.1 Firestore Collections

Cloud Firestore is used as the database. It is a NoSQL document-based database organised into collections. This project uses two collections: users and projects.

5.2 users Collection

Document ID = Firebase Auth UID (exact match required for profile lookup)

Field	Type / Description
uid	string — Firebase Auth UID (set programmatically, not stored in doc)
name	string — Full name, e.g. Dr. Ayesha Malik
email	string — Login email address
role	string — Either admin or faculty
gender	string — Male or Female
department	string — e.g. Software Engineering

designation	string — Professor, Associate Professor, etc.
phone	string — Contact number
joinedAt	Timestamp — Account creation date
profileComplete	boolean — Whether profile setup is done

5.3 Projects Collection

Document ID = auto-generated by Firestore (addDoc)

Field	Type / Description
title	string — Project title
supervisor	string — Faculty name (denormalised for display)
supervisorId	string — Firebase Auth UID of supervisor (used for filtering)
coSupervisor	string — Co-supervisor name or None
students	array<string> — List of student names (2-4)
studentCount	number — Length of students array
industrialPartner	string — Company name or None
sdg	string — e.g. SDG 4, SDG 11
status	string — pending, under_review, accepted, or rejected
uploadedBy	string — UID of admin who created the record
uploadedAt	Timestamp — Creation timestamp
updatedAt	Timestamp — Last update timestamp

5.4 Key Design Decisions

- supervisorId stores the Firebase Auth UID, not the name, enabling reliable filtering even if the name changes.
- The supervisor's name is denormalised (stored alongside supervisorId) to avoid extra lookups on display.
- Firestore document ID for users equals the Firebase Auth UID to enable O(1) profile lookup without queries.
- experimentalForceLongPolling is enabled in Firestore init to resolve connectivity issues on high-latency networks.

6. Role-Based Access Control (RBAC)

The system enforces two roles: admin and faculty. Access is controlled at three levels: routing, UI rendering, and data filtering.

6.1 Feature Matrix

Feature	Admin	Faculty
View all projects	Yes	No
View own projects only	Yes	Yes
Add new project	Yes	No
Change project status	Yes	No
Delete project	Yes	No
View all faculty	Yes	No
Add faculty account	Yes	No
Edit faculty profile	Yes	No
Delete faculty	Yes	No
View faculty detail page	Yes	No
View personal dashboard stats	Yes	Yes

6.2 Data Filtering

For faculty users, the ProjectsTable component receives supervisorId = user.uid as a prop. The component filters all fetched projects client-side, showing only projects where the project.supervisorId === supervisorId. This ensures faculty can never see projects assigned to other supervisors.

7. Key Components

7.1 AuthContext

A React Context that wraps the entire application. It exposes three values: user (the full UserProfile or null), loading (boolean), and logout (async function). All protected pages consume this context via the useAuth() hook.

7.2 ProjectsTable

A reusable table component used on both admin and faculty project pages. It accepts two props:

- isAdmin: boolean controls whether status dropdowns and delete buttons are shown
- supervisorId?: string when provided, filters projects to only show this faculty's records

The component fetches all projects from Firestore on mount, then applies client-side filtering via useMemo for performance.

7.3 FacultyTable

An admin-only component that lists all faculty members. It supports inline editing of name, department, designation, and phone. Changes are saved to Firestore via updateDoc on blur or save action. Delete triggers deleteDoc with a confirmation dialogue.

7.4 Seed Page (/admin/seed)

A developer utility page that populates the database with realistic sample data. It creates 4 faculty accounts in Firebase Auth, writes their profiles to Firestore, and creates 10 sample projects distributed among the faculty. The admin session is preserved and restored during faculty creation to avoid session hijacking by the new accounts.

8. Project Status Workflow

Every project has a status field that moves through a defined lifecycle. Only admins can change project status.

Status	Meaning
pending	Project submitted, awaiting admin review
under_review	Admin is actively reviewing the project
accepted	Project approved and confirmed
rejected	Project-rejected faculty may need to revise

> **Flow: pending → under_review → accepted OR pending → rejected**

Status changes are persisted to Firestore via updateDoc immediately. The UI updates optimistically without requiring a page reload.

9. Firestore Data Operations

9.1 Read Operations

Operation	Usage
getDocs(collection)	Fetch all documents — projects list, faculty list
getDoc(doc)	Fetch single document — user profile by UID, faculty by ID
query + where	Filter projects by supervisorId on the faculty detail page

9.2 Write Operations

Operation	Usage
addDoc	Add a new project to the projects collection
setDoc	Create a user profile with a specific document ID (= Auth UID)
updateDoc	Update project status, update faculty profile fields
deleteDoc	Delete project, delete faculty profile

9.3 Network Configuration

Firestore is initialised with experimentalForceLongPolling: true. This setting forces Firestore to use long-polling instead of WebSocket streaming, which significantly improves reliability on high-latency or restricted networks (e.g. Pakistani ISPs). This was a critical fix that resolved intermittent connection failures during development.

10. Security Considerations

10.1 Current Firestore Rules

The current Firestore security rules are configured for development:

```
allow read, write: if true;
```

WARNING: These rules allow any unauthenticated user to read/write all data. Must be updated before production.

10.2 Recommended Production Rules

```
match /users/{userId} {
  allow read: if request.auth != null;
  allow write: if request. auth.uid == userId;
}
match /projects/{projectId} {
  allow read: if request.auth != null;
  allow write: if request.auth != null;
}
```

10.3 Authentication Security

- Firebase Auth handles all password hashing and token management.
- JWTs are automatically refreshed by the Firebase SDK.
- Admin role enforcement happens server-side via Firestore rules (in production).
- Google OAuth uses industry-standard PKCE flow via Firebase.

11. Environment & Deployment

11.1 Required Environment Variables

Variable	Description
NEXT_PUBLIC_FIREBASE_API_KEY	Firebase project API key
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	Firebase auth domain
NEXT_PUBLIC_FIREBASE_PROJECT_ID	Firestore project ID
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET	Firebase storage bucket
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	FCM sender ID
NEXT_PUBLIC_FIREBASE_APP_ID	Firebase app ID

11.2 Local Development

```
git clone <repo>
npm install
cp .env.example .env.local   # fill in Firebase config
npm run dev                   # starts on localhost:3000
```

11.3 Production Deployment

The application is optimised for deployment on Vercel. Set all NEXT_PUBLIC_ environment variables in the Vercel project settings dashboard. No server configuration is required. Next.js handles SSR and static optimisation automatically.

12. Known Issues & Future Work

12.1 Current Limitations

- Firestore security rules are open must be locked down before any production deployment.
- No pagination on project or faculty tables may slow down with large datasets (100+ records).
- No email notifications when project status changes.
- Faculty cannot view their own profile or update their information.
- No file upload support for project proposals or documents.

12.2 Planned Enhancements

- Server-side role verification using Firebase Admin SDK in Next.js API routes.
- Firestore pagination using startAfter() for large datasets.
- Email notifications via Firebase Cloud Functions when the project status is updated.
- Student portal as a third role.
- PDF export of project reports.
- Audit log tracks who changed what and when.

13. Glossary

Term	Definition
FYP	Final Year Project, the capstone project for graduating students
Auth UID	Unique identifier assigned by Firebase Auth to every user
Firestore	Google Cloud NoSQL document database used as the app backend
App Router	Next.js 13+ routing system based on the /app directory

AuthContext	React Context providing global authentication state
supervisorId	The Auth UID of the faculty member supervising a project
RBAC	Role-Based Access Control restricting features by user role
SSR	Server-Side Rendering: pages rendered on the server before sending to the client
DXA	Document Exchange: Absolute units used in DOCX formatting (1440 = 1 inch)